

Indice

Position Paper — Why an Inline Network Encryptor Cannot Replace the MP-QUIC/STRIPES Crypto Layer, and How NQX Should Integrate Instead

1

1

2

2

2

2

3

3

3

3

3

4

5

Position Paper — Why an Inline Network Encryptor Cannot Replace the MPQUIC/STRIPES Crypto Layer, and How NQX Should Integrate Instead

Document ID	TPZ-STRIPES-CAL-POS-001
Title	Response to NQX PoC RFI — Architectural Position on Inline Network Encryption
Version	1.0
Date	2026-06-27
Classification	Restricted — Authorised Technical Partners Only
Issuing Entity	Telespazio S.p.A. — Engineering Division
Related document	STRIPES_External_Crypto_Provider_Spec.md (TPZ-STRIPES-CAL-EXT-001) — the authoritative integration contract referenced throughout this paper

1. Purpose of this document

Your RFI asks us to describe network diagrams, VLANs, subnets, routing domains, and node counts so that an NQX appliance can be deployed **inline, downstream of the MPQUIC node**, transparently encrypting the physical links it sits on.

Before we can answer those questions meaningfully, we need to flag that **the deployment model implied by the RFI is structurally incompatible with how our system is built**, and would not deliver the security or availability properties the PoC is meant to validate. This document explains why, in technical terms, and then describes the integration point that does work: a **software cryptographic provider plugged into our Crypto Abstraction Layer (CAL)**, not a network appliance inserted into the traffic path.

We are not rejecting NQX as a cryptographic engine — we are explaining where its cryptography needs to live for it to do any good in this architecture.

2. What our system actually is — the part that matters for this decision

Our transport layer (internally named **STRIPES**) does not move traffic over a single network link. It takes IP traffic from one local interface and **bonds it simultaneously across several independent physical paths** — for example a GEO satellite terminal, a LEO satellite modem, and an LTE/terrestrial uplink, all active at the same time, all monitored continuously, with traffic shifted away from any path that degrades or fails.

Two properties of this design are the reason an inline appliance cannot do what you are proposing:

1. **There is no single “channel” to sit on.** The thing protecting availability is the fact that paths are independent, dynamically used, and can come and go (a modem can drop and reappear with a different IP, a satellite terminal can hand off, etc.). “The channel” downstream of our VM is not one Ethernet segment with a fixed L2/L3 design — it is N physically and logically independent UDP flows, whose number and identity change at runtime.
2. **Encryption is not applied to “the link” — it is applied per unit of data, with metadata that only our software has.** Every individual unit of data we send out (we call it a *shard*) is sealed with a fresh nonce and authenticated against a 24-byte context that includes, among other fields, which physical path it is going out on, its sequence number, its QoS class, and the current key epoch. This binding is what stops an adversary from replaying, reordering across paths, or splicing shards between links — attacks that are specific to multipath bonding and do not exist in a single-link topology. **This context is constructed by our transport core at the moment a shard is about to leave on a specific path.** Nothing downstream of that point — including a hardware appliance sitting on the wire — has visibility into it.

The reference document attached to this paper ([STRIPES_External_Crypto_Provider_Spec.md](#)) describes this contract in full and is what governs every cryptographic integration we accept, regardless of vendor.

3. Why “NQX downstream of the VM, encrypting the channel” does not work

3.1 One appliance cannot sit on N independent, dynamic paths

An inline L2/L3 appliance is designed to be inserted into **one** fixed network segment between two endpoints. Our node has multiple physical uplinks active concurrently, and the set of active uplinks is not static — it is exactly what our path-liveness and failover logic manages in software. To replicate today’s design with NQX you would need **one appliance, with its own keys and tunnel, per physical path**, kept in sync with whichever paths are currently up. That is not a PoC detail to tune — it is a different architecture, with N times the hardware, N times the operational overhead, and N new single points of failure exactly where we are trying to remove them.

3.2 The cryptographic binding cannot be reproduced outside the transport core

As described above, every shard is authenticated against per-shard context (path identifier, sequence number, QoS class, epoch) that is generated by our transport core at send time. An appliance receiving already-formed UDP datagrams on the wire has no access to this context — it can encrypt the datagram as an opaque blob, but it cannot reproduce or verify the binding that protects against cross-path replay and splicing. If NQX encryption is added **on top of** our own (double encryption of an opaque blob), it adds CPU cost and latency without adding any verifiable security property beyond what our AEAD already provides. If NQX encryption is meant to **replace** ours, the protections described in §2 disappear, because they live in metadata an inline appliance cannot construct.

3.3 The session and key lifecycle is driven by the application, not by the wire

Our session keys are not static pre-shared secrets. They are derived from a QUIC/TLS handshake (via the TLS Exporter) and are subject to scheduled re-keying — the live epoch transitions are coordinated between our client and server cores, and our AEAD operations are epoch-aware. An appliance sitting passively on the wire has no way to participate in this handshake or to learn when a re-key has occurred without being made an active party to the QUIC session itself — at which point it is no longer a transparent inline encryptor, it is a TLS-terminating man-in-the-middle, which breaks end-to-end authentication between our two endpoints.

3.4 Hot-path performance budget

Our crypto layer seals and opens shards in-process, with a documented requirement of **zero heap allocation and sub-10-microsecond latency per shard**, at rates exceeding 100,000 shards/second on a single high-throughput link. Routing traffic through an external appliance adds a network hop, serialization, and queuing on every path — on satellite links, where round-trip time is already in the hundreds of milliseconds and every millisecond of added jitter degrades the loss-recovery logic that keeps the bonded tunnel stable, this is not a negligible overhead.

3.5 Summary

What the RFI assumes	What is actually true in our system
One stable network path/segment to insert an appliance into	N independent, dynamically-appearing/disappearing physical paths
Encryption can be applied to “the link”	Encryption is applied per data unit, bound to per-unit metadata generated by the application
Keys are static/provisioned out of band	Keys are derived per session from a live QUIC/TLS handshake and rotated on a schedule
An appliance can be transparent on the wire	Anything that needs the session key material must be an active party inside the handshake, not a passive bump-in-the-wire
Latency budget allows an extra inline hop	Hot path budget is sub-10 μ s in-process; SATCOM RTT is already the dominant constraint

None of this is a reflection on NQX’s cryptographic quality. It is a statement about **where** that cryptography needs to be invoked from for it to do anything useful in a multipath-bonded SATCOM transport.

4. How the integration must work instead: NQX as a software provider behind our Crypto Abstraction Layer

Our transport core does not hard-code its cryptography. It is built against an internal **Crypto Abstraction Layer (CAL)** — a Ports & Adapters boundary — so that a certified third-party cipher suite or key-exchange algorithm can be substituted **without any change to the transport core**, while all the multipath-specific logic described in §2-§3 stays exactly where it needs to be: inside our software, immediately before/after a shard hits the wire.

Concretely:

- The integration artefact is a **Go plugin** (.so, built with `go build -buildmode=plugin`), loaded once at process startup, **running in the same process and address space as our transport core**. There is no network hop, no IPC, no protocol translation — a call into NQX’s cryptography is an ordinary in-process function call.

- NQX can integrate at one of three levels, depending on what it wants to expose:
 - **Level A — AEAD only:** NQX supplies `Seal/Open` (e.g. a certified AES-256-GCM or other AEAD implementation); we continue to manage key exchange, nonce construction, AAD, and epoch/session lifecycle.
 - **Level B — Key exchange only:** NQX supplies key generation and session-key derivation (e.g. a certified or post-quantum KEX); we continue to manage the AEAD operations and per-shard binding.
 - **Level C — Full provider:** NQX supplies both, end to end.
- The shard-level context (nonce, 24-byte AAD, epoch handling) is **always constructed by our core**, as documented in `STRIPES_External_Crypto_Provider_Spec.md` §6–§7. The NQX plugin receives exactly what it needs to seal/open or derive keys — it never needs visibility into path scheduling, loss recovery, or failover logic, which remain entirely out of scope for the integration and proprietary to us.
- If NQX’s cryptographic engine is not natively Go, this is not a blocker: the plugin’s outward-facing shell must be Go, but it is permitted to call into a C/C++/Rust library, or link a pre-certified FIPS 140-3 module, via CGO. This is explicitly documented as Option 2/3 in §P.6.1 of the attached specification.
- Hot-path requirements (thread-safety, zero heap allocation per call, no blocking I/O, no logging of key material, no `panic`) are non-negotiable and apply regardless of integration level — they are listed in full in §10–§11 of the attached specification.

This is the same integration model we require of every external cryptographic partner on this program; it is not a constraint introduced specifically for NQX.

5. Re-reading your RFI against this model

Most of the questions in your RFI are framed around inserting a physical appliance into a network path. Reframed against the integration model in §4, here is what we actually need to scope a PoC:

Your original question	What it becomes under the software-provider model
Network diagrams, sites/links to be protected, L2 vs L3 design, VLANs/subnets/routing domains	Not applicable — there is no network segment for NQX to be inserted into. There is one integration point: the CAL plugin interface, loaded in-process on our existing MPQUIC nodes.
Number of NQX nodes for the PoC	Not applicable as physical nodes. Relevant unit is: one software plugin instance per MPQUIC process, on each of our existing nodes (client side and server side).
Availability / failover requirements for the appliance	Not applicable — failover of physical paths is already handled by our transport core. NQX’s plugin only needs to be stateless/restart-safe per the lifecycle in the spec (<code>Init</code> , <code>Close</code> , idempotency).
Management network requirements	Not applicable — there is no separate management plane; configuration is a YAML file read by <code>Init(configPath)</code> on our node.
Type of traffic/protocols	Already fixed: IP traffic over UDP-encapsulated shards; this is data our core hands to the plugin already shaped for <code>Seal/Open</code> (or raw key material for KEX) — see §6 of the spec.

The questions we **do** need answered from your engineering team are:

1. Can NQX's cryptographic engine (AEAD and/or KEX) be exposed as a Go-callable library — pure Go, or via CGO against a C/C++/Rust/FIPS module — meeting the hot-path constraints in §10 of the attached spec (goroutine-safe, zero allocation per `Seal/Open` call, sub-10µs latency, no blocking I/O)?
2. Which specific algorithms/cipher suites are you proposing (e.g. is this a classical AES-GCM implementation, a post-quantum KEM, both)?
3. What certification do you hold for the proposed algorithms (FIPS 140-3, Common Criteria, national scheme), and can you provide test vectors per §12.1 of the spec?
4. Is your team able to deliver and support a Go plugin artefact (linux/amd64 and linux/arm64), accepting the toolchain alignment constraints in §9 of the spec?

If the answer to (1) is no — if NQX's cryptography genuinely cannot be invoked as an in-process software call and only exists as appliance firmware — then it is not a fit for this integration point, independent of its cryptographic merit, and we should say so early rather than scope a PoC around a deployment model that cannot be adopted.

6. Recommended next steps

1. Share `STRIPES_External_Crypto_Provider_Spec.md` with NQX's engineering team and ask them to confirm, against §4–§5 of that document, which integration level (A/B/C) they can realistically support and in what timeframe.
2. Get a written answer to the four questions in §5 above before any network-facing PoC planning continues.
3. Once NQX confirms it can deliver a Go-callable software provider, we will provide: the exact Go toolchain version, the build/Docker image for CGO alignment (if needed), and the YAML configuration profile (`crypto.profile: custom_provider`) to activate it on a test instance.

Document end — TPZ-STRIPES-CAL-POS-001 v1.0 — 2026-06-27 Telespazio S.p.A. — Restricted Distribution — Authorised Technical Partners Only